

Assignment Report

Name: Divya Prakash Pandey

Roll Number: 240369

Assignment-1: Memory-Efficient Versioned File Indexer Using C++

1. Introduction / Overview of the Problem

The goal of this assignment was to build a memory-efficient text analyzer in C++ that can process very large log files. Since the files can be very large, the main restriction was that the program should not load the whole file into memory.

Instead, the file needs to be read in small chunks using a fixed-size buffer. While reading the file, the program builds a word-frequency index that stores how many times each word appears in the file.

This index is then used to answer three types of queries:

- Finding the frequency of a specific word
 - Finding the difference in frequency of a word between two versions
 - Displaying the top-K most frequent words
-

2. Design and Approach

I designed the program using object-oriented programming so that different parts of the task are handled by separate classes. This makes the code easier to understand and modify.

The overall workflow of the program is:

1. Read the file in small chunks using a fixed-size buffer.
2. Extract words from the buffer.
3. Convert words to lowercase so that matching is case-insensitive.
4. Store the word frequencies for each version of the file.
5. Run different queries on the stored index.

Since the datasets are large, I tried to keep the program efficient by avoiding unnecessary memory allocations and using fast file reading methods.

3. Description of the Classes Used

The program is divided into a few main classes, each responsible for a specific task.

BufferedReader

This class handles reading the file from disk. Instead of reading the entire file at once, it reads fixed-size chunks of data using a buffer. This ensures that memory usage stays within the required limits.

Tokenizer

The tokenizer processes the characters read from the file and extracts valid words. A word is defined as a sequence of alphanumeric characters. It also converts uppercase characters to lowercase so that word comparisons are case-insensitive.

VersionedIndexer

This class stores the word frequency information for each version of the file. Internally, it uses an `unordered_map` where the key is the word and the value is the number of times the word appears.

QueryProcessor

This is an abstract base class that defines a common interface for executing queries. Three classes inherit from it:

- **WordCountQuery** – returns the frequency of a word in a version
- **DiffQuery** – finds the difference in frequency of a word between two versions
- **TopKQuery** – displays the K most frequent words in a version

Using inheritance and polymorphism allows the program to execute different queries using a common interface.

4. How the File is Processed Using the Fixed-Size Buffer

To make sure the entire file is never loaded into memory, the program reads the file chunk by chunk.

The process works like this:

1. A buffer of size between 256 KB and 1024 KB is allocated based on the command line argument.
2. The program reads a block of data into this buffer using `fread`.
3. The tokenizer scans the buffer and extracts words from it.
4. If a word gets split between two chunks (for example "err" at the end of one chunk and "or" at the beginning of the next), the unfinished part is stored temporarily.
5. When the next chunk is read, the leftover part is attached to it so the word can be reconstructed correctly.

At the end of the file, a small cleanup step ensures that the last unfinished word is also counted.

5. Assumptions and Observations

While implementing the program, I made a few observations that helped improve performance.

- **Pre-allocating map memory:**
When inserting a large number of words into an `unordered_map`, the map may repeatedly resize itself. This can slow down the program. To reduce this overhead, I reserved space in advance using `reserve()`.

- **Case conversion:**
Since the program processes millions of characters, repeatedly calling functions like `tolower()` inside the loop can slow things down. A simple ASCII conversion approach was used instead.
- **Safe map lookups:**
Using the `[]` operator on an `unordered_map` can insert new elements unintentionally when a key does not exist. To avoid this, the `.find()` method is used during queries so that the index structure is not modified.